

WinForms Development Guidelines

These are the coding and design guidelines and instructions for WinForms Expert Agent development. When customer asks/requests will require the creation of new projects

New Projects: * Prefer .NET 10+. Note: MVVM Binding requires .NET 8+. * Prefer `Application.SetColorMode(SystemColorMode.System);` in `Program.cs` at application startup for DarkMode support (.NET 9+). * Make Windows API projection available by default. Assume 10.0.22000.0 as minimum Windows version requirement.

```
<TargetFramework>net10.0-windows10.0.22000.0</TargetFramework>
```

Critical:

□ **NUGET:** New projects or supporting class libraries often need special NuGet packages. Follow these rules strictly:

- Prefer well-known, stable, and widely adopted NuGet packages - compatible with the project's TFM.
- Define the versions to the latest STABLE major version, e.g.: [2.*,)

⚙ **Configuration and App-wide HighDPI settings:** *app.config* files are discouraged for configuration for .NET. For setting the HighDpiMode, use e.g. `Application.SetHighDpiMode(HighDpiMode.SystemAware)` at application startup, not *app.config* nor *manifest* files.

Note: SystemAware is standard for .NET, use PerMonitorV2 when explicitly requested.

VB Specifics: - In VB, do NOT create a *Program.vb* - rather use the VB App Framework. - For the specific settings, make sure the VB code file *ApplicationEvents.vb* is available. Handle the `ApplyApplicationDefaults` event there and use the passed `EventArgs` to set the App defaults via its properties.

Property	Type	Purpose
ColorMode	SystemColorMode	DarkMode setting for the application. Prefer System. Other options: Dark, Classic.
Font	Font	Default Font for the whole Application.

Property	Type	Purpose
HighDpiMode	HighDpiMode	SystemAware is default. PerMonitorV2 only when asked for HighDPI Multi-Monitor scenarios.

❑ Critical Generic WinForms Issue: Dealing with Two Code Contexts

Context	Files/Location	Language Level	Key Rule
Designer Code	<i>.designer.cs</i> , inside <code>InitializeComponent</code>	Serialization-centric (assume C# 2.0 language features)	Simple, predictable, parsable
Regular Code	<i>.cs</i> files, event handlers, business logic	Modern C# 11-14	Use ALL modern features aggressively

Decision: In *.designer.cs* or `InitializeComponent` → Designer rules. Otherwise → Modern C# rules.

❑ Designer File Rules (TOP PRIORITY)

⚠ Make sure Diagnostic Errors and build/compile errors are eventually completely addressed!

❑ Prohibited in `InitializeComponent`

Category	Prohibited	Why
Control Flow	<code>if</code> , <code>for</code> , <code>foreach</code> , <code>while</code> , <code>goto</code> , <code>switch</code> , <code>try/catch</code> , <code>lock</code> , <code>await</code> , VB: <code>On Error/Resume</code>	Designer cannot parse

Category	Prohibited	Why
Operators	? : (ternary), ??/?./?[] (null coalescing/conditional), nameof()	Not in serialization format
Functions	Lambdas, local functions, collection expressions (...=[] or ...=[1,2,3])	Breaks Designer parser
Backing fields	Only add variables with class field scope to ControlCollections, never local variables!	Designer cannot parse

Allowed method calls: Designer-supporting interface methods like SuspendLayout, ResumeLayout, BeginInit, EndInit

❑ Prohibited in *.designer.cs* File

- ❑ Method definitions (except InitializeComponent, Dispose, preserve existing additional constructors)
- ❑ Properties
- ❑ Lambda expressions, DO ALSO NOT bind events in InitializeComponent to Lambdas!
- ❑ Complex logic
- ❑ ??/?./?[] (null coalescing/conditional), nameof()
- ❑ Collection Expressions

❑ Correct Pattern

- ❑ File-scope namespace definitions (preferred)

❑ Required Structure of InitializeComponent Method

Order	Step	Example
1	Instantiate controls	button1 = new Button();
2	Create components container	components = new Container();
3	Suspend layout for container(s)	SuspendLayout();
4	Configure controls	Set properties for each control

Order	Step	Example
5	Configure Form/UserControl LAST	ClientSize, Controls.Add(), Name
6	Resume layout(s)	ResumeLayout(false);
7	Backing fields at EOF	After last #endregion after last method.

(Try meaningful naming of controls, derive style from existing code-base, if possible.)

```
private void InitializeComponent()
{
    // 1. Instantiate
    _picDogPhoto = new PictureBox();
    _lblDogographerCredit = new Label();
    _btnAdopt = new Button();
    _btnMaybeLater = new Button();

    // 2. Components
    components = new Container();

    // 3. Suspend
    ((ISupportInitialize)_picDogPhoto).BeginInit();
    SuspendLayout();

    // 4. Configure controls
    _picDogPhoto.Location = new Point(12, 12);
    _picDogPhoto.Name = "_picDogPhoto";
    _picDogPhoto.Size = new Size(380, 285);
    _picDogPhoto.SizeMode = PictureBoxSizeMode.Zoom;
    _picDogPhoto.TabStop = false;

    _lblDogographerCredit.AutoSize = true;
    _lblDogographerCredit.Location = new Point(12, 300);
    _lblDogographerCredit.Name = "_lblDogographerCredit";
    _lblDogographerCredit.Size = new Size(200, 25);
    _lblDogographerCredit.Text = "Photo by: Professional Dogographer";

    _btnAdopt.Location = new Point(93, 340);
    _btnAdopt.Name = "_btnAdopt";
    _btnAdopt.Size = new Size(114, 68);
    _btnAdopt.Text = "Adopt!";

    // OK, if BtnAdopt_Click is defined in main .cs file
```

```

        _btnAdopt.Click += BtnAdopt_Click;

        // NOT AT ALL OK, we MUST NOT have Lambdas in InitializeComponent!
        _btnAdopt.Click += (s, e) => Close();

        // 5. Configure Form LAST
        AutoScaleDimensions = new.SizeF(13F, 32F);
        AutoScaleMode = AutoScaleMode.Font;
        ClientSize = new.Size(420, 450);
        Controls.Add(_picDogPhoto);
        Controls.Add(_lblDogographerCredit);
        Controls.Add(_btnAdopt);
        Name = "DogAdoptionDialog";
        Text = "Find Your Perfect Companion!";
        ((ISupportInitialize)_picDogPhoto).EndInit();

        // 6. Resume
        ResumeLayout(false);
        PerformLayout();
    }

#endregion

// 7. Backing fields at EOF

private PictureBox _picDogPhoto;
private Label _lblDogographerCredit;
private Button _btnAdopt;

```

Remember: Complex UI configuration logic goes in main .cs file, NOT .designer.cs.



Modern C# Features (Regular Code Only)

Apply **ONLY** to .cs files (event handlers, business logic).
NEVER in .designer.cs or InitializeComponent.

Style Guidelines

Category	Rule	Example
Using directives	Assume global	System.Windows.Forms, System.Drawing, Sys- tem.ComponentModel
Primitives	Type names	int, string, not Int32, String
Instantiation	Target-typed	Button button = new();
prefer types over var	var only with obvious and/or awkward long names	var lookup = ReturnsDictOf- StringAndListOfTu- ples() // type clear
Event handlers	Nullable sender	private void Handler(object? sender, EventArgs e)
Events	Nullable	public event EventHandler? MyEvent;
Trivia	Empty lines before return/code blocks	Prefer empty line before
this qualifier	Avoid	Always in NetFX, otherwise for disambiguation or extension methods
Argument validation	Always; throw helpers for .NET 8+	ArgumentNullException.ThrowIfNull(control)
Using statements	Modern syntax	using frmOptions modalOptionsDlg = new(); // Always dispose modal Forms!

Property Patterns (⚠ CRITICAL - Common Bug Source!)

Pattern	Behavior	Use Case	Memory
=> new Type()	Creates NEW instance EVERY access	⚠ LIKELY MEMORY LEAK!	Per-access allocation

Pattern	Behavior	Use Case	Memory
{ get; } = new()	Creates ONCE at construction	Use for: Cached/constant	Single allocation
=> _field ?? Default	Computed/dynamic value	Use for: Calculated property	Varies

// ❌ *WRONG - Memory leak*

```
public Brush BackgroundBrush => new SolidBrush(BackColor);
```

// ✅ *CORRECT - Cached*

```
public Brush BackgroundBrush { get; } = new SolidBrush(Color.White);
```

// ✅ *CORRECT - Dynamic*

```
public Font CurrentFont => _customFont ?? DefaultFont;
```

Never “refactor” one to another without understanding semantic differences!

Prefer Switch Expressions over If-Else Chains

// ✅ *NEW: Instead of countless IFs:*

```
private Color GetStateColor(ControlState state) => state switch
{
    ControlState.Normal => SystemColors.Control,
    ControlState.Hover => SystemColors.ControlLight,
    ControlState.Pressed => SystemColors.ControlDark,
    _ => SystemColors.Control
};
```

Prefer Pattern Matching in Event Handlers

// *Note nullable sender from .NET 8+ on!*

```
private void Button_Click(object? sender, EventArgs e)
{
    if (sender is not Button button || button.Tag is null)
        return;

    // Use button here
}
```

When designing Form/UserControl from scratch

File Structure

Language	Files	Inheritance
C#	FormName.cs + Form- Name.Designer.cs	Form or UserControl
VB.NET	FormName.vb + Form- Name.Designer.vb	Form or UserControl

Main file: Logic and event handlers

Designer file: Infrastructure, constructors, Dispose, Initialize-Component, control definitions

C# Conventions

- File-scoped namespaces
- Assume global using directives
- NRTs OK in main Form/UserControl file; forbidden in code-behind .designer.cs
- Event *handlers*: object? sender
- Events: nullable (EventHandler?)

VB.NET Conventions

- Use Application Framework. There is no Program.vb.
- Forms/UserControls: No constructor by default (compiler generates with InitializeComponent() call)
- If constructor needed, include InitializeComponent() call
- CRITICAL: Friend WithEvents controlName as ControlType for control backing fields.
- Strongly prefer event handlers Subs with Handles clause in main code over AddHandler in fileInitializeComponent

Classic Data Binding and MVVM Data Binding (.NET 8+)

Breaking Changes: .NET Framework vs .NET 8+

Feature	.NET Framework <= 4.8.1	.NET 8+
Typed DataSets Object Binding	Designer supported Supported	Code-only (not recommended) Enhanced UI, fully supported

Feature	.NET Framework <= 4.8.1	.NET 8+
Data Sources Window	Available	Not available

Data Binding Rules

- Object DataSources: INotifyPropertyChanged, BindingList<T> required, prefer ObservableCollection from MVVM CommunityToolkit.
- ObservableCollection<T>: Requires BindingList<T> a dedicated adapter, that merges both change notifications approaches. Create, if not existing.
- One-way-to-source: Unsupported in WinForms DataBinding (workaround: additional dedicated VM property with NO-OP property setter).

Add Object DataSource to Solution, treat ViewModels also as DataSources

To make types as DataSource accessible for the Designer, create .datasource file in Properties\DataSources\:

```
<?xml version="1.0" encoding="utf-8"?>
<GenericObjectDataSource DisplayName="MainViewModel" Version="1.0"
  xmlns="urn:schemas-microsoft-com:xml-msdatasource">
  <TypeInfo>MyApp.ViewModels.MainViewModel, MyApp.ViewModels, Version=1.0.0.0, C
</GenericObjectDataSource>
```

Subsequently, use BindingSource components in Forms/UserControls to bind to the DataSource type as “Mediator” instance between View and ViewModel. (Classic WinForms binding approach)

New MVVM Command Binding APIs in .NET 8+

API	Description	Cascading
Control.DataObjectProperty	Bindable property for MVVM	Yes (down hierarchy)
ButtonBase.Command	Command binding	No
ToolStripItem.Command	Command binding	No
*.CommandParameter	Argument passed to command	No

Note: ToolStripItem now derives from BindableComponent.

MVVM Pattern in WinForms (.NET 8+)

- If asked to create or refactor a WinForms project to MVVM, identify (if already exists) or create a dedicated class library for ViewModels based on the MVVM CommunityToolkit
- Reference MVVM ViewModel class library from the WinForms project
- Import ViewModels via Object DataSources as described above
- Use new `Control.DataContext` for passing ViewModel as data sources down the control hierarchy for nested Form/UserControl scenarios
- Use `Button[Base].Command` or `ToolStripItem.Command` for MVVM command bindings. Use the `CommandParameter` property for passing parameters.
- - Use the `Parse` and `Format` events of `Binding` objects for custom data conversions (`IValueConverter` workaround), if necessary.

```
private void PrincipleApproachForIValueConverterWorkaround()
{
    // We assume the Binding was done in InitializeComponent and look up
    // the bound property like so:
    Binding b = text1.DataBindings["Text"];

    // We hook up the "IValueConverter" functionality like so:
    b.Format += new ConvertEventHandler(DecimalToCurrencyString);
    b.Parse += new ConvertEventHandler(CurrencyStringToDecimal);
}

• Bind property as usual.
• Bind commands the same way - ViewModels are Data Sources!
  Do it like so:

// Create BindingSource
components = new Container();
mainViewModelBindingSource = new BindingSource(components);

// Before SuspendLayout
mainViewModelBindingSource.DataSource = typeof(MyApp.ViewModels.MainViewModel);

// Bind properties
_txtDataField.DataBindings.Add(new Binding("Text", mainViewModelBindingSource, "

// Bind commands
```

```
_tsmFile.DataBindings.Add(new Binding("Command", mainViewModelBindingSource, "To
_tsmFile.CommandParameter = "File";
```

WinForms Async Patterns (.NET 9+)

Control.InvokeAsync Overload Selection

Your Code Type	Overload	Example Scenario
Sync action, no return	InvokeAsync(Action)	Update label.Text
Async operation, no return	InvokeAsync(Func<Task>)	Load data + update UI
Sync function, returns T	InvokeAsync<T>(Func<T>)	Get control value
Async operation, returns T	InvokeAsync<T>(Func<Task<T>>)	Async work + result

⚠ Fire-and-Forget Trap

```
// ❌ WRONG - Analyzer violation, fire-and-forget
await InvokeAsync<string>(() => await LoadDataAsync());

// ✅ CORRECT - Use async overload
await InvokeAsync<string>(async (ct) => await LoadDataAsync(ct), outerCancellation)
```

Form Async Methods (.NET 9+)

- ShowAsync(): Completes when form closes. Note that the IAsyncState of the returned task holds a weak reference to the Form for easy lookup!
- ShowDialogAsync(): Modal with dedicated message queue

CRITICAL: Async EventHandler Pattern

- All the following rules are true for both [modifier] void async EventHandler(object? s, EventArgs e) as for overridden virtual methods like async void OnLoad or async void OnClick.
- async void event handlers are the standard pattern for WinForms UI events when striving for desired asynch implementation.
- CRITICAL: ALWAYS nest await MethodAsync() calls in try/catch in async event handler — else, YOU'D RISK CRASHING THE PROCESS.

Exception Handling in WinForms

Application-Level Exception Handling

WinForms provides two primary mechanisms for handling unhandled exceptions:

AppDomain.CurrentDomain.UnhandledException: - Catches exceptions from any thread in the AppDomain - Cannot prevent application termination - Use for logging critical errors before shutdown

Application.ThreadException: - Catches exceptions on the UI thread only - Can prevent application crash by handling the exception - Use for graceful error recovery in UI operations

Exception Dispatch in Async/Await Context

When preserving stack traces while re-throwing exceptions in async contexts:

```
try
{
    await SomeAsyncOperation();
}
catch (Exception ex)
{
    if (ex is OperationCanceledException)
    {
        // Handle cancellation
    }
    else
    {
        ExceptionDispatchInfo.Capture(ex).Throw();
    }
}
```

Important Notes: - `Application.OnThreadException` routes to the UI thread's exception handler and fires `Application.ThreadException`.
- Never call it from background threads — marshal to UI thread first.
- For process termination on unhandled exceptions, use `Application.SetUnhandledExceptionMode(UnhandledExceptionMode.ThrowException)` at startup. - **VB Limitation:** VB cannot await in catch block. Avoid, or work around with state machine pattern.

CRITICAL: Manage CodeDOM Serialization

Code-generation rule for properties of types derived from `Component` or `Control`:

Approach	Attribute	Use Case	Example
Default value	[DefaultValue]	Simple types, no serialization if matches default	[DefaultValue(typeof(Color), "Yellow")]
Hidden	[DesignerSerializationVisibility.Hidden]	Runtime only data	Calculated properties
Conditional	ShouldSerialize* + Reset*()	Complex conditions	Custom fonts, optional settings

```

public class CustomControl : Control
{
    private Font? _customFont;

    // Simple default - no serialization if default
    [DefaultValue(typeof(Color), "Yellow")]
    public Color HighlightColor { get; set; } = Color.Yellow;

    // Hidden - never serialize
    [DesignerSerializationVisibility(DesignerSerializationVisibility.Hidden)]
    public List<string> RuntimeData { get; set; }

    // Conditional serialization
    public Font? CustomFont
    {
        get => _customFont ?? Font;
        set { /* setter logic */ }
    }

    private bool ShouldSerializeCustomFont()
        => _customFont is not null && _customFont.Size != 9.0f;

    private void ResetCustomFont()
        => _customFont = null;
}

```

Important: Use exactly ONE of the above approaches per property for types derived from Component or Control.

WinForms Design Principles

Core Rules

Scaling and DPI: - Use adequate margins/padding; prefer TableLayoutPanel (TLP)/FlowLayoutPanel (FLP) over absolute positioning of controls. - The layout cell-sizing approach priority for TLPs is: * Rows: AutoSize > Percent > Absolute * Columns: AutoSize > Percent > Absolute

- For newly added Forms/UserControls: Assume 96 DPI/100% for AutoScaleMode and scaling
- For existing Forms: Leave AutoScaleMode setting as-is, but take scaling for coordinate-related properties into account
- Be DarkMode-aware in .NET 9+ - Query current DarkMode status: `Application.IsDarkModeEnabled`
 - Note: In DarkMode, only the `SystemColors` values change automatically to the complementary color palette.
- Thus, owner-draw controls, custom content painting, and Data-GridView theming/coloring need customizing with absolute color values.

Layout Strategy

Divide and conquer: - Use multiple or nested TLPs for logical sections - don't cram everything into one mega-grid. - Main form uses either SplitContainer or an "outer" TLP with % or AutoSize-rows/cols for major sections. - Each UI-section gets its own nested TLP or - in complex scenarios - a UserControl, which has been set up to handle the area details.

Keep it simple: - Individual TLPs should be 2-4 columns max - Use GroupBoxes with nested TLPs to ensure clear visual grouping. - RadioButton cluster rule: single-column, auto-size-cells TLP inside AutoGrow/AutoSize GroupBox. - Large content area scrolling: Use nested panel controls with AutoScroll-enabled scrollable views.

Sizing rules: TLP cell fundamentals - Columns: * AutoSize for caption columns with `Anchor = Left | Right`. * Percent for content columns, percentage distribution by good reasoning, `Anchor = Top | Bottom | Left | Right`. Never dock cells, always anchor! * Avoid *Absolute* column sizing mode, unless for unavoidable fixed-size content (icons, buttons). - Rows: * AutoSize for rows with "single-line" character (typical entry fields, captions, checkboxes). * Percent for multi-line TextBoxes, rendering areas AND filling distance filler for

remaining space to e.g., a bottom button row (OK|Cancel). * Avoid *Absolute* row sizing mode even more.

- Margins matter: Set Margin on controls (min. default 3px).
- Note: Padding does not have an effect in TLP cells.

Common Layout Patterns

Single-line TextBox (2-column TLP) Most common data entry pattern: - Label column: AutoSize width - TextBox column: 100% Percent width - Label: Anchor = Left | Right (vertically centers with TextBox) - TextBox: Dock = Fill, set Margin (e.g., 3px all sides)

Multi-line TextBox or Larger Custom Content - Option A (2-column TLP)

- Label in same row, Anchor = Top | Left
- TextBox: Dock = Fill, set Margin
- Row height: AutoSize or Percent to size the cell (cell sizes the TextBox)

Multi-line TextBox or Larger Custom Content - Option B (1-column TLP, separate rows)

- Label in dedicated row above TextBox
- Label: Dock = Fill or Anchor = Left
- TextBox in next row: Dock = Fill, set Margin
- TextBox row: AutoSize or Percent to size the cell

Critical: For multi-line TextBox, the TLP cell defines the size, not the TextBox's content.

Container Sizing (CRITICAL - Prevents Clipping)

For GroupBox/Panel inside TLP cells: - MUST set `AutoSize = true` and `AutoSizeMode = GrowOnly` - Should Dock = Fill in their cell - Parent TLP row should be AutoSize - Content inside GroupBox/Panel should use nested TLP or FlowLayoutPanel

Why: Fixed-height containers clip content even when parent row is AutoSize. The container reports its fixed size, breaking the sizing chain.

Modal Dialog Button Placement

Pattern A - Bottom-right buttons (standard for OK/Cancel): - Place buttons in FlowLayoutPanel: `FlowDirection = RightToLeft` -

Keep additional Percentage Filler-Row between buttons and content.
 - FLP goes in bottom row of main TLP - Visual order of buttons: [OK] (left) [Cancel] (right)

Pattern B - Top-right stacked buttons (wizards/browsers): -
 Place buttons in FlowLayoutPanel: FlowDirection = TopDown - FLP
 in dedicated rightmost column of main TLP - Column: AutoSize - FLP:
 Anchor = Top | Right - Order: [OK] above [Cancel]

When to use: - Pattern A: Data entry dialogs, settings, confirmations
 - Pattern B: Multi-step wizards, navigation-heavy dialogs

Complex Layouts

- For complex layouts, consider creating dedicated UserControls for logical sections.
- Then: Nest those UserControls in (outer) TLPs of Form/UserControl, and use DataContext for data passing.
- One UserControl per TabPage keeps Designer code manageable for tabbed interfaces.

Modal Dialogs

Aspect	Rule
Dialog buttons	Order -> Primary (OK): AcceptButton, DialogResult = OK / Secondary (Cancel): CancelButton, DialogResult = Cancel
Close strategy	DialogResult gets applied by DialogResult implicitly, no need for additional code
Validation	Perform on <i>Form</i> , not on Field scope. Never block focus-change with CancelEventArgs.Cancel = true

Use DataContext property (.NET 8+) of Form to pass and return modal data objects.

Layout Recipes

Form Type	Structure
MainForm	MenuStrip, optional ToolStrip, content area, StatusStrip
Simple Entry Form	Data entry fields on largely left side, just a buttons column on right. Set meaningful Form MinimumSize for modals
Tabs	Only for distinct tasks. Keep minimal count, short tab labels

Accessibility

- CRITICAL: Set AccessibleName and AccessibleDescription on actionable controls
- Maintain logical control tab order via TabIndex (A11Y follows control addition order)
- Verify keyboard-only navigation, unambiguous mnemonics, and screen reader compatibility

TreeView and ListView

Control	Rules
TreeView	Must have visible, default-expanded root node
ListView	Prefer over DataGridView for small lists with fewer columns
Content setup	Generate in code, NOT in designer code-behind
ListView columns	Set to -1 (size to longest content) or -2 (size to header name) after populating
SplitContainer	Use for resizable panes with TreeView/ListView

DataGridView

- Prefer derived class with double buffering enabled
- Configure colors when in DarkMode!
- Large data: page/virtualize (VirtualMode = True with Cell-ValueNeeded)

Resources and Localization

- String literal constants for UI display NEED to be in resource files.
- When laying out Forms/UserControls, take into account that localized captions might have different string lengths.
- Instead of using icon libraries, try rendering icons from the font "Segoe UI Symbol".
- If an image is needed, write a helper class that renders symbols from the font in the desired size.

Critical Reminders

#	Rule
1	InitializeComponent code serves as serialization format - more like XML, not C#
2	Two contexts, two rule sets - designer code-behind vs regular code
3	Validate form/control names before generating code
4	Stick to coding style rules for InitializeComponent
5	Designer files never use NRT annotations
6	Modern C# features for regular code ONLY
7	Data binding: Treat ViewModels as DataSources, remember Command and CommandParameter properties